

Homework 01

Justin Adrowski

Problem 1:

A) Identify the (concrete) benefits and drawbacks (at least one of each and three total) of establishing a hardware abstraction that uses simple, primitive instructions rather than implementing a high-level programming language directly.

- (1) The biggest benefit of a simple, primitive hardware abstraction ISA like RISC-V is that the instruction sets make it easier to design efficient, fast hardware.
- (2) Another benefit to simple, primitive instruction sets that they make it easier for compilers to generate code for assemblers to take and translate into binary
- (3) One drawback to these types of ISAs is that you can get more instructions per program. In a high-level program designed for hardware with specific instructions tailored toward that program, this means extra unnecessary instructions and lost optimization.

B) RISC-V: an example of ISA design decisions. Using the specifications for the ISA (https://drive.google.com/file/d/1uviu1nH-tScFfgrovvFCrj7Omv8tFtkp/view?usp=drive_link) and ABI (https://drive.google.com/file/d/1Ja_Tpp_5Me583CGVD-BIZMlgGBnlKU4R/view?usp=drive_link), answer the following questions and cite the page(s) in the spec you used to answer the question:

- i. What is the XLEN for RV32I?

2.1. Programmers' Model for Base Integer ISA

Table 2 shows the unprivileged state for the base integer ISA. For RV32I, the 32 x registers are each 32 bits wide, i.e., XLEN=32. Register x0 is hardwired with all bits equal to 0. General purpose registers x1-x31 hold values that various instructions interpret as a collection of Boolean values, or as two's complement signed binary integers or unsigned binary integers.

This is from page 23 of the manual, section 2.1

- ii. How many general purpose integer registers are there for RV32I? Why 32 and not 8 or 16? Why not 64? I.e., what are the trade-offs of changing the number of architectural registers?
 - Q1: From the same section as (i): RV32I has 31 general purpose integer registers, [x1,x31] – pg 23, table 2

- Q2: From page 23 of manual, section 2.1

performance, and energy consumption. Although 16 registers would arguably be sufficient for an integer ISA running compiled code, it is impossible to encode a complete ISA with 16 registers in 16-bit instructions using a 3-address format.

This statement describes why not 16 or fewer like 8. Regarding 32, a larger number of registers helps performance on high-performance code and allows for a complete ISA instruction encoding with the 16-bit instructions with 3-address format.

- Q3: Regarding 64, there is no explicit statement why not, but if 64 registers were used, it would require 6 bits to determine address, which would eat up more space of a 32-bit instruction word. I'd assume also that the extra registers would cost too much space and power vs. the extra bit of performance they provide to warrant their use. Also, more registers is slower.

- iii. Register x0 has a special use. What is it? Is this enforced by a hardware implementation (i.e., microarchitecture), the ISA, the ABI, or the assembler?

Q1: x0 is always 0 no matter what. We learned this in class.

Q2: It is also specified in section 2.1 on pg 23 of the manual that it is hardwired, implying hardware enforced implementation. It is also specified by the ISA on pg 41 that, "x0 is a dedicated zero register"

- iv. The PC is a register in the architectural state. What is the purpose of the PC? Can you use the add instruction to modify it?

Q1: It holds the address of the current instruction from section 2.1, pg 23

Q2: No, add cannot be used to modify the PC.

- The `AUIPC` instruction, which adds a 20-bit upper immediate to the `PC`, replaces the `RDNPC` instruction, which only read the current `PC` value. This results in significant savings for position-independent code.

This is from pg 29, sec. 2.4.1 of the manual. `AUIPC` is what modifies the PC. The `add` function is an instruction where the address of it would need to be determined by the PC, so it itself can't modify the pc.

- v. Does a HW implementation of RV32I need to identify overflow for any of the base instructions? Why not?

A RISC-V hart has a single byte-addressable address space of 2^{XLEN} bytes for all memory accesses. A *word* of memory is defined as 32 bits (4 bytes). Correspondingly, a *halfword* is 16 bits (2 bytes), a *doubleword* is 64 bits (8 bytes), and a *quadword* is 128 bits (16 bytes). The memory address space is circular, so that the byte at address $2^{XLEN} - 1$ is adjacent to the byte at address zero. Accordingly, memory address computations done by the hardware ignore **overflow** and instead wrap around modulo 2^{XLEN} .

This if from pg 17 of the manual, sec 1.4

*We did not include special instruction-set support for **overflow** checks on integer arithmetic operations in the base instruction set, as many overflow checks can be cheaply implemented using RISC-V branches. Overflow checking for unsigned addition requires only a single additional branch instruction after the addition: add t0, t1, t2; bltu t0, t1, overflow.*

This sc comes from pg 28, sec. 2.4 of the manual. As it says, the base instructions don't need overflow checks since they can be cheaply implemented with RISC-V branches.

- vi. Priming for a later callback: Why were branches with integrated arithmetic comparisons included in the ISA?

As seen from this screenshot (sc) of pg. 33, the branches with integrated arithmetic comparisons fit into a regular pipeline, avoid additional condition code state or use of a temp. reg, & reduce static code size and dynamic instruction fetch traffic.

motivated by the observation that a combined compare-and-branch instruction fits into a regular pipeline, avoids additional condition code state or use of a temporary register, and reduces static code size and dynamic instruction fetch traffic. Another

Problem 2: RISC-V

(a) Identify three products not mentioned in lecture videos that use a RISC-V-based processor. Cite the reference and why you believe it.

- 1- Espressif ESP32-C3 / ESP32-C6: Popular single-core and multi-core Wi-Fi/Bluetooth MCUs built on 32-bit RISC-V architecture – [muxup.com/elektor magazine](https://muxup.com/elektor-magazine)
- 2- Raspberry Pi RP2350: MCU featured on Raspberry Pi Pico 2 – has dual RISC-V hazard3 co-processing options alongside standard ARM cores – [elektor magazine](https://www.elektor-magazine.com)
- 3- SERV: world's smallest RISC-V CPU, optimized for ultra-low resource usage – the chip letter

For all three, I believe them since I asked google about the websites and organizations credibility level, and google rated them as credible.

(b) RISC-V does not have a clear instruction (clr dst). The intent of this operation is to set the dst register to 0. Why would such a simple, basic instruction not be included in RISC-V? List three arithmetic or logical instructions or instruction sequences not mentioned during lecture that produce the same effect as a clr instruction.

Since x0 in RISC-V is always 0, it can be used to set other registers to 0 and effectively clear them. Also, there are other ways to do it arithmetically.

1. Addi t0, x0, 0
2. Li t0, 0
3. Ori t0, x0, 0

(c) What does the following program do (give a description in English sentences)?
xor x1, x1, x2
xor x2, x1, x2
xor x1, x1, x2

Xor x1, x1, x2 sets $x1 = x1 \oplus x2$ (if either only x1 or x2 is one, x1 is set to 1, 0 otherwise)

Xor x2, x1, x2 sets $x2 = x1 \oplus x2$ (if either only x1 or x2 is one, x2 is set to 1, 0 otherwise)

Xor x1, x1, x2 sets $x1 = x1 \oplus x2$ (if either only x1 or x2 is one, x1 is set to 1, 0 otherwise)

So, after three instructions, say original inputs are $x1 = a$, $x2 = b$

$X1 = x1 \text{ xor } x2 \Rightarrow x1 = x1 \text{ xor } (x1 \text{ xor } x2) \Rightarrow x1 = (a \text{ xor } b) \text{ xor } ((a \text{ xor } b) \text{ xor } b)$, which comes to $x1 = b$ after drawing out a truth table.

And $x2 = (a \text{ xor } b) \text{ xor } b$, which comes to $x2 = a$.

So, the final result with original inputs $x1 = a$, $x2 = b$, is $x1 = b$, $x2 = a$. This means it simply swapped the inputs.

(d) Assume that int variables a, b, c, and d are mapped to registers s0, s1, s2, and s3, respectively. Write a RISC-V program that implements: $a = (((b-7) / 2) \& c) > (d \mid 0xF00)$; Use only add, sub, addi, and, andi, or, ori, sll, slli, srl, srli, sra, srai, slt, and slti instructions. Since b, c, and d must not be overwritten, use as many temporary registers (t0-t6) as needed.

Addi t0, s1, -7

Srai t0, t0, 1

And t0, t0, s2

Li t1, 0xF00

Or t1, s3, t1

Slt s0, t1, t0

Problem 3: RARS Introduction

(a) does RARS simulate the ABI, ISA, and/or microarchitecture/implementation of a RISC-V processor? Explain. Where would you look for error messages? Where would you look for text output from a running program? Where would you enter text as input? Where would you be able to find the contents of memory location 0x10010004? [We'll use this version of RARS since it is used for testing the RISC-V processors you will design for your term project.]

RARS simulates the ISA, not the microarchitecture bc there are countless processors that run a RISC-V ISA instruction set with different hardware. It also does execute whatever ABI binary instructions are written, but gives no notion as to whether those instructions would “break” the hardware, like using sp as a scratch reg without restoring it.

Errors show up in the messages pane at the bottom. Text output appears in the Run I/O tab. Text to read in as input is also in this tab. The data segment view in execute tab is where 0x10010004's contents would be found. It'd be the 2nd word after base address.

(b) Describing prob3.s functionality and implementing in C

Prob3.s basically declares an array of 11 integers called vars, prompts a user to input 2 numbers, one for arithmetic and one to choose an index, adds the first input to the integer in the array at the chosen index, and then prints the new integer value at that index in the array.

(c) RARS edit to find array min

I verified correctness by storing the third index input address location before arithmetic on S1, then using that value after finding the min and storing it in the location to load a word from that location into a0 before printing it out. I also looked at the input index values for each case and confirmed the correct min was chosen and stored in the third location.

```
Enter an array index1:
2
Enter an array index2:
4
Enter an array index3 (where to store the minimum):
6
4
-- program is finished running (0) --

Reset: reset completed.

Enter an array index1:
5
Enter an array index2:
10
Enter an array index3 (where to store the minimum):
7
25
-- program is finished running (0) --

Reset: reset completed.

Enter an array index1:
0
Enter an array index2:
6
Enter an array index3 (where to store the minimum):
8
25
-- program is finished running (0) --
```

RARS I/O window after editing script